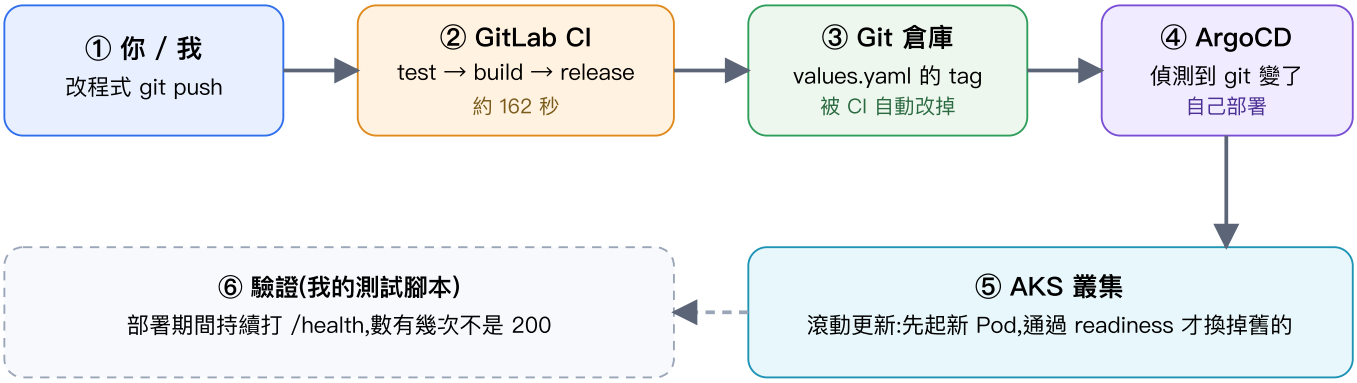
Day 23:改一行程式,自己上線

端到端驗證:從 `git push` 到新版本在 AKS 上服務,中間我完全不動手。

一句話: Day 22 把「CI 自動寫回 image tag」接起來,Day 23 就是把整條線跑好幾遍,證明它**穩定**、**不用人**,並把跑的時候暴露出來的真問題修掉。

整條自動化流水線



實測時間(第 7–9 輪)

輪次	CI(push → tag 寫回 git)	全程(push → 新版本就緒)	部署期間非 200 次數
7	162 秒	294 秒	0
8	162 秒	296 秒	0
9	162 秒	354 秒	0

CI 之後到部署完成多出的 1–3 分鐘,是 ArgoCD 要「發現 git 變了」。它是定期去 git 看(預設約每 3 分鐘),不是 git 通知它。我的測試中**沒有手動 refresh**,等的就是它自己的節奏。

每個環節到底誰做什麼

環節	誰負責	做的事	為什麼這樣設計
test	GitLab CI	跑 21 個 pytest + `helm lint` / `helm template`	壞程式、壞 chart 在最前面就擋掉
build	GitLab CI(arm64 runner)	建 backend / frontend image,tag = commit 短 SHA,推到 GitLab registry	AKS 節點是 ARM,image 架構要一致,否則 `exec format error`
release	GitLab CI	把 `mychart/values.yaml` 兩處 tag 改成這次 SHA,commit 訊息 `release <SHA> [skip ci]`,push 回 main	CI **不碰 cluster**,只改 git。cluster 長什麼樣,唯一依據是 git
部署	ArgoCD	發現 git 與 cluster 不同 → 自動 sync	selfHeal:手動改 cluster 會被改回 git 的樣子

image tag 就是版本號(也是整條線的「接力棒」)



好處:看 cluster 跑的 image tag,就能直接對回是哪個 commit;git 歷史就是部署歷史,要回滾就是 `git revert` 那筆 release commit。

為什麼 release commit 要帶 `[skip ci]`

這個 commit 是 CI 自己推的。如果它又觸發一輪 pipeline,pipeline 又會推新的 release commit……會**無限循環**。GitLab 看到 `[skip ci]` 就不跑 pipeline。

為什麼壞測試不會被部署

stage 順序是 test → build → release。前面任一關失敗,release 就不會執行,tag 沒被改,ArgoCD 沒有新東西可部署。(Day 24 實測過:壞測試推上去,338 秒沒有 release commit,cluster 版本沒變。)

Day 23 實際發生了什麼(誠實版)

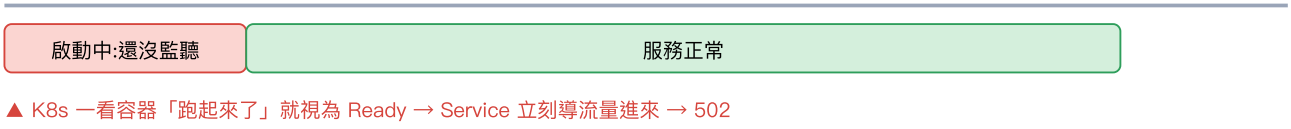
計畫要求「連續 3 次都成功」。我一共嘗試了 4 組,前 3 組都有問題,每個都是真實缺陷。

組別	結果	發生什麼	根因
第 1 組	全失敗	三輪的 `git push` 全被 GitLab 拒絕,根本沒進 CI	我的腳本用 `git pull`,預設拉的是 GitHub(origin);CI 的 release commit 只在 GitLab,本機永遠落後一筆
第 2 組	1 過 2 敗	第 1、2 輪部署完立刻打 `/health` 得 **502**;第 3 輪正常	Deployment **沒有 readiness probe**
第 3 組(加了 probe)	仍不穩	第 4 輪結尾 502 一次;第 5 輪部署期間 1 次非 200;第 6 輪 0 次	Pod 關閉時 ingress 還在送請求(沒有優雅下線)
第 4 組(加了 preStop)	3/3 通過	第 7、8、9 輪部署期間非 200 都是 0 次	—

要誠實說的:第 4 組只有 3 輪、每輪只在部署期間取樣幾次,「0 次失敗」跟修法一致,但這是小樣本,**不是統計上的證明**。面試時說「連續 3 次零失敗」可以,說「保證零停機」就過頭了。

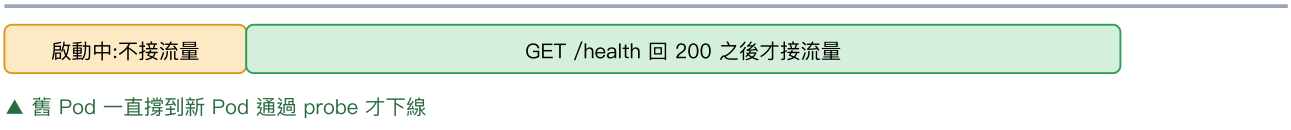
問題一:沒有 readiness probe → 部署完打到 502

沒有 readiness probe(之前)



▲ K8s 一看容器「跑起來了」就視為 Ready → Service 立刻導流量進來 → 502

有 readiness probe(現在)



▲ 舊 Pod 一直撐到新 Pod 通過 probe 才下線

為什麼之前沒發現:只有 1 個 Pod 的時候,沒有人在滾動更新的瞬間去打它。Day 23 的測試腳本剛好在部署一完成就打,才抓到。

加了什麼:backend 探測 `GET /health`(每 3 秒),frontend 探測 `/`;另外加 liveness(起來 15 秒後每 10 秒檢查,卡死就重啟)。

問題二:Pod 關閉時 ingress 還在送請求

Pod 被刪除那一刻,K8s 同時做兩件事(互不等待)

A. 從 Service / Ingress 名單移除
要傳播到 ingress-nginx,需要一點時間

B. 對容器送 SIGTERM(開始關閉)
uvicorn 馬上停止接新連線

B 比 A 快 → 這段空檔 ingress 仍把請求送給「正在關的 Pod」→ 502

修法:preStop 先睡 5 秒,再送 SIGTERM
這 5 秒裡名單更新已傳播完,ingress 不再送新請求;進行中的請求也能收尾

這是我推測的原因。證據是「加了 preStop 之後 3 輪都 0 次失敗」,但我沒有直接抓到 ingress 的 log 證明,所以是「最可能的解釋 + 結果一致」,不是確診。

問題三:git 兩個遠端,CI 的 commit 讓本機落後



我的 `git pull --rebase` 沒指定遠端,預設拉到 GitHub,所以永遠拉不到 CI 的 commit。改成 `git pull --rebase gitlab main` 就解決。**這是工作流程的教訓:CI 會往 main 寫東西,push 前一定要先同步。**

我處理得不漂亮的地方:分岔後我一度想用 `git reset --hard` 對齊,被系統擋下(會丟掉本機 commit)。我改用 `git merge` 接回去,沒有改寫歷史、沒有強推。結果歷史裡多了幾個 merge commit,有點亂,但沒有資料遺失。

Day 24 補充:故意推壞測試,確認被擋

步驟	結果
推一個 `assert 1 == 2` 的測試(`61ab3dbf`)	CI 的 test 階段失敗
等 338 秒(正常 release 約 160 秒就會出現)	**沒有** `release 61ab3dbf` commit
檢查 cluster	版本仍是 `d452fc30`,Pod 沒被換
刪掉壞測試再推	新版本 `cc37a119` 自動上線,`/health` 200

我看不到 GitLab 的 pipeline 畫面,是用「release commit 沒出現 + cluster 版本沒變」推斷 test 失敗擋住了發版。你可以自己開 GitLab 截一張紅色 test 階段的圖放 README,證據更直接。

面試可以問 / 可以講的點

Q1. 你的 CI/CD 流程是什麼?

> 「push 之後 GitLab CI 跑測試、建 arm64 image、再把新 image 的 tag 寫回 git 裡 Helm 的 values。ArgoCD 偵測到 git 變了,自己部署到 AKS。CI 本身完全不碰 cluster,所以 cluster 的狀態永遠以 git 為準。我實測連續 3 次,CI 約 162 秒,全程 5 到 6 分鐘,中間沒有手動步驟。」

Q2. 為什麼 CI 不直接 kubectl apply / helm upgrade?

> 「兩個原因。第一,CI 要有 cluster 的憑證,風險大;現在 CI 只需要寫 git 的權限。第二,GitOps 要求『git 是唯一真相』:有人手動改 cluster,ArgoCD 的 selfHeal 會改回來,所有變更都留在 git 歷史裡可審計、可回滾。」

Q3. 怎麼回滾?

> 「`git revert` 那筆 release commit,ArgoCD 就把 tag 換回舊的。因為每個版本 tag 都是 commit SHA、image 都還在 registry,回滾就是改回舊 tag。」

Q4. 你怎麼知道壞版本不會被部署?

> 「stage 有順序,test 失敗 release 就不跑,tag 不會被寫回 git。我實際推過一個必定失敗的測試,等了超過正常時間兩倍,沒有 release commit,cluster 版本也沒變。」

Q5. 做端到端測試時發現了什麼問題?(最好的故事)

> 「部署完立刻打 health 偶爾 502。我查出 Deployment 沒有 readiness probe,K8s 以為容器起來就 Ready。補上 probe 後還有零星失敗,我推測是 Pod 關閉時 ingress 名單還沒更新,就加了 preStop sleep 做優雅下線,之後連跑 3 輪都是 0 次失敗。重點是:**流程跑起來不等於零停機,要用測試去量。**」

Q6. 這條流程還有什麼不足?(誠實回答加分)

> 「第一,兩個 pipeline 同時結束時 release 階段會競爭,可能把 tag 蓋回舊版本,應該用 GitLab 的 `resource_group` 串行化,我還沒做。第二,沒有 staging 環境,也沒有部署後自動 smoke test 與自動回滾。第三,我沒做統計上有意義的停機測試,只有 3 輪的小樣本。」

別說錯的話

❌「保證零停機」→ 只有小樣本證據。 ❌「ArgoCD 會即時部署」→ 它是定期輪詢(預設約 3 分鐘),我的測試等的就是這個。 ❌「CI 部署到 cluster」→ CI 只改 git,部署是 ArgoCD 做的。